

Weave: Fine-Grained Dynamic SM Scheduling in an MoE Megakernel for Compute-Communication Overlap

Ziyu Huang^{*1}, Yangjie Zhou^{*2}, Chenhao Zhu¹, Zihan Liu¹, Jinyu Liu¹, Shulai Zhang¹, Xingxun Tang¹, Hongzhe Yan¹, Xinhao Luo¹, Minyi Guo¹, Xiu Lin³, Yinghao Yu³, Guodong Yang³, Liping Zhang³, Shixuan Sun¹, Jingwen Leng¹

¹Shanghai Jiao Tong University
Shanghai, China

²National University of Singapore
Singapore

³Alibaba Group
China

huang_ziyu@sjtu.edu.cn

Abstract

Mixture-of-Experts (MoE) inference under expert parallelism (EP) turns each MoE layer into a distributed computation with costly dispatch and combine communication. State-of-the-art systems reduce this cost through communication-computation overlap, splitting the GPU’s SMs for communication and computation respectively. However, this approach still leaves GPU resources wasted along two dimensions. Spatially, the best SM split is determined by each layer’s routing result and varies across layers and GPUs, so fixed policies mismatch the workload and waste either NVLink bandwidth or compute throughput. Temporally, complex MoE data dependencies introduce bubbles that leave SMs idle.

We present **Weave**, to our knowledge the first MoE overlap system that performs fine-grained dynamic SM scheduling—deciding per layer and per GPU by routing results at runtime. Once routing completes, each layer’s communication and computation volumes become known; Weave exploits this predictability through a lightweight cost model running inside the persistent megakernel: a spatial scheduler partitions SMs into communication workers and computation workers to match the communication/computation throughput ratio, and a temporal scheduler coordinates the two worker groups to minimize SM idleness. On 4×H100 SXM GPUs across six mainstream MoE models, Weave achieves a 2.89× geometric-mean MoE-layer speedup and a 1.33× geometric-mean end-to-end speedup over five state-of-the-art baselines.

CCS Concepts: • Software and its engineering → Software performance.

Keywords: MoE, Expert Parallelism, communication-computation overlap, GPU SM scheduling

1 Introduction

Mixture-of-Experts (MoE) scales large language models by replacing dense feed-forward networks with sparsely activated expert networks [22]. Each token is routed to only a small subset of experts, increasing model capacity while keeping per-token computation nearly constant. As modern MoE models grow to tens or hundreds of experts [3, 6, 12, 15, 18, 20, 21], distributed inference commonly relies on expert parallelism (EP), where experts are partitioned across GPUs. Under EP, tokens are routed to experts placed on different GPUs, turning each MoE layer into a distributed computation with cross-GPU token movement.

This distributed execution makes dispatch and combine a central cost of MoE inference. In each MoE layer, dispatch sends token activations to the GPUs that host their selected experts; after expert computation, combine returns and reduces the expert outputs into final token representations. Prior work reports that dispatch and combine can consume up to 40% of MoE layer latency [28]. This pressure is amplified by hardware scaling: as shown in Figure 1, BF16 tensor throughput has grown faster than NVLink bandwidth across recent GPU generations, and this widening compute-interconnect gap makes communication an increasingly important component of MoE layer latency.

To reduce this exposed communication cost, software-level communication-computation overlap has become critical for efficient MoE inference. Based on their awareness of the dynamic routing results, existing systems fall into three categories. Figure 2 compares the execution timelines of these approaches.

(i) **No explicit SM partitioning** Triton-Distributed (TD) [32] runs communication and computation serially across all SMs, relying on global preemption for overlap at operator boundaries. All SMs execute dispatch, GEMM, and

^{*}Both authors contributed equally to this work.

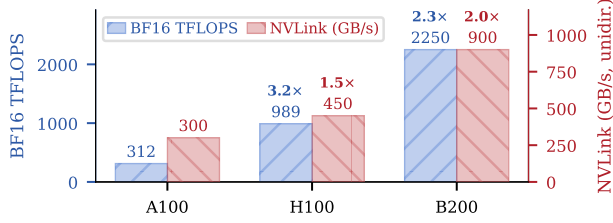


Figure 1. GPU compute vs. NVLink bandwidth scaling across A100/H100/B200. Left blue axis: BF16 dense TFLOPS. Right red axis: NVLink unidirectional GB/s.

combine in sequence, yielding minimal communication-computation overlap.

(ii) **Static SM partition** DeepEP [30] and ParallelKittens (PK) [26] fix the number of communication SMs at compile time, applying the same partition to all layers. Dedicating separate SM groups to communication and computation enables overlap, but data dependencies in the communication-computation pipeline (up gemm cannot begin until dispatch delivers tokens; combine must wait for down gemm to finish) create pipeline bubbles that leave SMs idle.

(iii) **Coarse-grained dynamic** Comet [28] selects the communication/computation SM ratio from a pre-profiled kernel library based on the input sequence length, switching once per iteration. This adapts to workload intensity across iterations, but the partition still does not respond to per-layer routing variation.

However, none of these approaches adapts the SM partition to the actual routing result of each layer, leaving substantial GPU resource waste along both spatial and temporal dimensions.

The waste manifests in two dimensions. Spatially, the optimal communication/computation SM ratio depends on each layer’s routing result, which determines the communication and computation volumes on each GPU. When the SM partition does not match these volumes, one group of SMs finishes early and idles while the other remains the bottleneck, wasting either communication bandwidth or computation throughput. Since routing skew varies across inputs, layers, and GPUs, a fixed or per-iteration partition inevitably mismatches some layers.

Temporally, even with a well-chosen SM split, the pipeline bubbles shown in Figure 2(b) persist: communication SMs remain idle during the middle computation phase, forming a large *mid bubble* that further degrades SM utilization.

Building on these observations, we propose **Weave**, a runtime SM scheduling system for MoE communication-computation overlap (Figure 2(d)). Weave exploits the fact that each layer’s communication and computation volumes become known after routing, and schedules SMs per layer and per GPU rather than relying on an offline fixed policy.

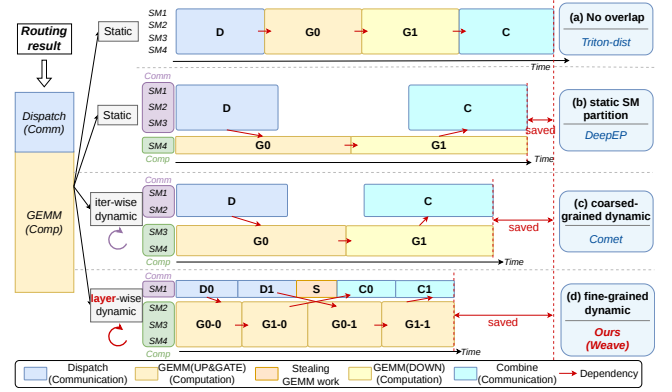


Figure 2. Four MoE execution patterns. (a) No overlap (TD): all SMs execute dispatch, GEMM, and combine serially. (b) Static SM partition (DeepEP): communication and computation SMs are fixed at compile time, leaving idle bubbles. (c) Coarse-grained dynamic (Comet): SM count adjusts per iteration. (d) Fine-grained dynamic (Weave): SM partition adjusts per layer and per GPU.

It combines a spatial scheduler that partitions SMs into communication workers and computation workers with a temporal scheduler that coordinates the two worker groups to minimize SM idleness, unified by a lightweight cost model running inside a persistent megakernel.

We implement Weave on PK [26] and evaluate it on 4xH100 SXM GPUs across six mainstream MoE architectures. Compared with five state-of-the-art baselines [26, 28, 30–32], Weave achieves the lowest per-layer MoE latency in all configurations, with a 2.89× geometric-mean MoE-layer speedup and a 1.33× geometric-mean end-to-end speedup.

This paper makes the following contributions.

- We identify routing-determined SM scheduling as a key inefficiency in distributed MoE inference, showing that the communication/computation SM split must be selected per layer and per GPU after routing completes.
- We design Weave’s runtime spatial scheduler, which uses routing-derived workload volumes and calibrated hardware throughput curves to choose the communication SM count for each layer and GPU.
- We design Weave’s temporal scheduler, which uses chunk pipelining and bubble stealing to reduce SM idleness caused by MoE data dependencies within each layer.
- We implement Weave based on Parallel Kittens [26] and show that it consistently improves MoE-layer and end-to-end inference latency over state-of-the-art baselines.

Our implementation will be open-sourced upon acceptance.

2 Background and Motivation

This section first introduces the basics of Mixture-of-Experts (MoE) and the corresponding computation/communication overlap techniques. Then we analyze the limitations of existing communication-related optimizations to derive our insights.

2.1 Mixture-of-Experts and Expert Parallelism

MoE replaces the monolithic FFN layer with tens to hundreds of smaller expert sub-networks, each activated only for a subset of tokens selected by a learned gating mechanism, thereby scaling model parameters while keeping per-token computation nearly constant.

The presence of multiple discrete experts gives rise to a new parallelism paradigm: expert parallelism (EP), under which experts are partitioned across GPUs. Hosting experts across multiple GPUs introduces cross-GPU communication overhead.

As illustrated in Figure 3, each MoE layer under EP consists of five stages: (1) **dispatch**, which sends each token to the GPU hosting its target experts; (2) **GEMM0**, the up-projection and gated FFN; (3) **activation**, typically a SiLU followed by element-wise multiplication; (4) **GEMM1**, the down-projection FFN; and (5) **combine**, which aggregates the per-expert results via weighted summation and returns them to the originating GPU. Among these, dispatch and combine are cross-GPU communication operations, while GEMM0, activation, and GEMM1 are local expert computations.

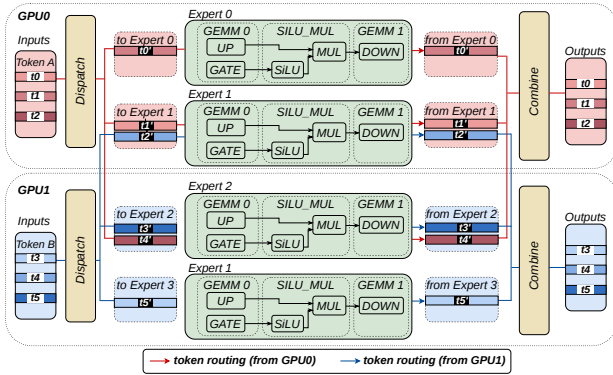


Figure 3. MoE forward pass under expert parallelism (2 GPUs, 4 experts, top- $k=3$ routing). Dispatch sends each token to the GPU hosting its expert; the expert sub-network sequentially executes gemm0 (UP+GATE), silu_mul, and gemm1 (DOWN); combine performs weighted summation and returns the results.

2.2 Compute-Communication Overlap

To hide dispatch and combine latency, modern systems overlap communication with computation by partitioning GPU

SMs between the two tasks; for example, DeepEP [30] dedicates 20 of the 132 SMs on an H100 to communication and uses the remaining 112 for computation.

However, even SM-level partitioning is a non-trivial design problem. We conduct an experiment that evaluates different compute-communication SM partitions across six MoE models (DeepSeek-V3 (DSv3), Phi-3.5-MoE, Qwen3-30B, Qwen3.5-35B, DeepSeek-V2-Lite (DSv2-Lite), DeepSeek-V2 (DSv2); seq = 8192 prefill, 4×H100 SXM, EP = 4, BF16). As shown in Figure 4(c), on the one hand, the optimal partition point that minimizes overall execution time varies across models, indicating that the SM partition must be adapted per layer. On the other hand, the trend itself also differs across models, suggesting that the strategy should adapt to different models, configurations, and other factors.

2.3 Limitations of SOTA Approaches

Existing distributed MoE systems either fix the number of communication SMs at compile time [26, 30], select from a pre-profiled configuration per iteration [28], or avoid explicit SM partitioning altogether and run communication and computation serially [32]. However, MoE workloads are *inherently dynamic*: the routing result of each layer changes, causing the communication and computation volumes to shift, and consequently the optimal communication SM count varies from layer to layer. Resolving this mismatch poses two key challenges: how to determine the optimal SM partition at runtime, and, once the partition is made, how to coordinate the two groups of SMs (comm workers and comp workers) to maximize overlap while minimizing idle time.

2.3.1 Challenge A: Runtime Partition Decision.

As discussed in §1, existing approaches fall into three suboptimal modes: static fixed, coarse-grained dynamic, and no overlap. We now quantify the performance gap on DSv2 (4×H100 SXM) with CodeSearchNet dataset. As the hatched bars of Figure 5 show, within a single MoE layer the per-GPU optimal c^* ranges from 12 to 36. Neither DeepEP’s fixed $c=20$ nor Comet’s per-iteration $c=18$ matches any GPU’s optimum. Since all GPUs synchronize after each MoE layer, the layer latency is determined by the slowest GPU. The hollow markers report the resulting per-GPU latency: on the bottleneck GPU (GPU 3), DeepEP is 1.33× slower than the per-GPU optimal, Comet 1.18×, and TD 1.44×.

Determining the optimal SM partition at runtime is therefore a key challenge.

2.3.2 Challenge B: Dynamic Coordination of Two Worker Groups.

Even after performing the SM partition described in Challenge A, complex dependencies between communication

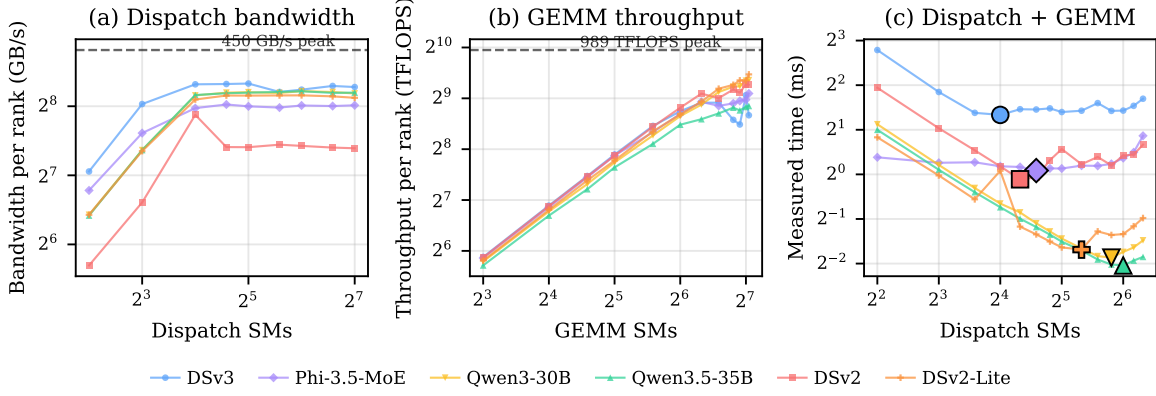


Figure 4. Impact of SM partitioning on compute-communication overlap (4×H100 SXM, 6 MoE models, seq = 8192 prefill, EP = 4, BF16). (a) Dispatch bandwidth vs. dispatch SM count c : near-linear then saturates. (b) GEMM throughput vs. compute SM count $N-c$: sub-linear under power and L2 cache constraints. (c) Dispatch-gemm total time with tile-level overlap as a function of c ; markers indicate each model’s optimal split.

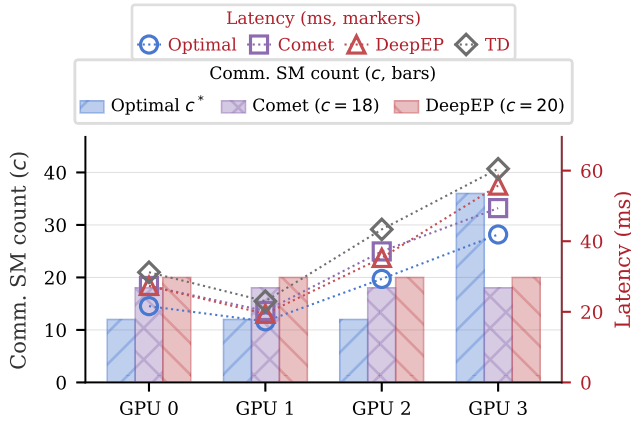


Figure 5. SM partition mismatch under existing policies (DSv2, CodeSearchNet dataset, Layer 1, 4×H100 SXM). Left y-axis (hatched bars): per-GPU optimal comm SM count c^* vs. Comet ($c=18$, per-iteration) and DeepEP ($c=20$, static); the optimal c^* ranges from 12 to 36 across GPUs. Right y-axis (hollow markers): per-GPU latency under each policy plus TD (serial); layer latency is limited by the slowest GPU.

and computation tasks make it difficult to coordinate the two worker groups so that they run in parallel (high comm/comp overlapping) without waiting for each other (high SM Active rate).

The MoE forward pass has inherent data dependencies: up gemm cannot begin until dispatch delivers the required tokens, and combine cannot start until down gemm finishes. Under SM partitioning, these dependencies cause communication workers to idle during the middle computation phase, forming pipeline bubbles that existing systems [26, 28, 30]

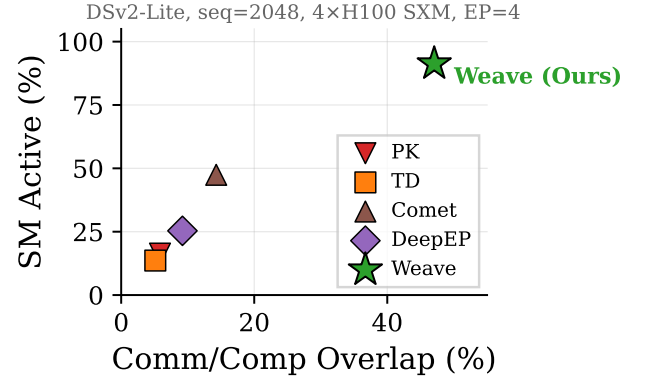


Figure 6. Performance of MoE overlap systems along two utilization dimensions: comm-comp overlap (x-axis) and SM Active (y-axis). Each marker is one system, measured on DSv2-Lite (seqlen = 2048, 4×H100 SXM, EP = 4, BF16).

do not fill. As shown in Figure 6, no existing system simultaneously achieves high SM Active and high comm-comp overlap; only Weave maintains high levels on both dimensions (91% SM Active, 47% overlap).

3 Design

In this section, we present the core design of Weave, a runtime SM scheduling system for efficient MoE execution under expert parallelism. Existing systems cannot perceive per-layer routing results and adapt the SM partition accordingly, which leads to missed optimization opportunities in two dimensions: spatially, a mismatched SM split wastes either communication bandwidth or compute throughput; temporally, the complex dependencies introduced

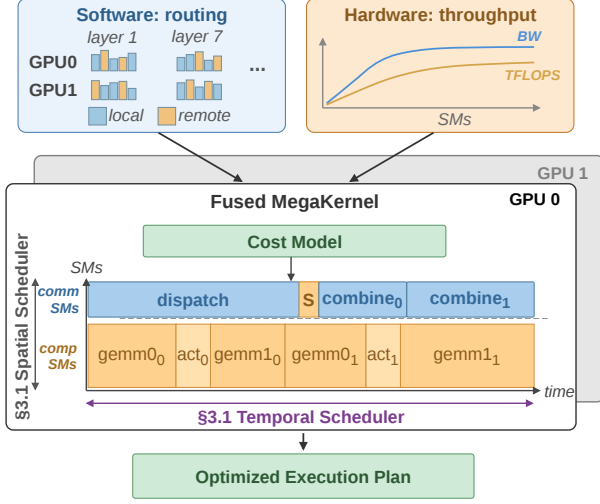


Figure 7. System overview of Weave. The system takes routing results (software input) and communication bandwidth / compute throughput curves (hardware input), and makes runtime decisions inside the megakernel along two dimensions: spatial scheduler (how to split SMs between communication and computation, §3.1), and temporal scheduler (how the two types of workers coordinate their execution, §3.2). The output is an optimized execution plan applied to the current layer.

by communication-computation overlap leave SMs idle. Weave addresses both dimensions with runtime scheduling.

Figure 7 gives an overview of Weave’s architecture. Weave takes two categories of input: software input, consisting of per-layer routing results that determine the communication volume and computation workload on each GPU; and hardware input, consisting of calibrated communication bandwidth and compute throughput curves as functions of SM count. These inputs are fed into a persistent megakernel that fuses all five forward operations of an MoE layer (dispatch, gemm0, silu_mul, gemm1, combine). Within this megakernel, each SM is assigned either a *comm* role (Dispatch and Combine) or a *comp* role (GEMM0, SiLU-Mul, and GEMM1). A lightweight cost model in the megakernel prologue jointly determines the spatial SM partition and the temporal execution order, producing a per-layer, per-GPU scheduling plan that coordinates the two types of workers throughout the layer.

The key observation behind Weave’s runtime scheduling is that the workload of an MoE layer becomes fully determined once routing completes. At that point, Weave knows

the number of tokens to send and receive, the token distribution across experts, and the resulting computation volume on each GPU. Weave therefore invokes its scheduler immediately after routing. The scheduler combines these runtime workload descriptors with hardware throughput profiles and uses a lightweight cost model inside the megakernel prologue to generate a layer-specific scheduling plan within microseconds. The megakernel then transitions to its main execution phase.

Weave jointly optimizes scheduling along two dimensions. The spatial scheduler (§3.1) selects the number of SMs assigned to communication so that communication bandwidth and computation throughput are balanced for the current layer. The temporal scheduler (§3.2) chunks routed tokens into pipelined microbatches and allows idle communication SMs to steal GEMM tiles when communication work is temporarily unavailable. Since the best temporal schedule depends on the throughput induced by the spatial SM split, Weave solves the two dimensions jointly using a unified cost model (§3.3).

Concretely, the scheduling decision is governed by two variables: the number of communication SMs c (how to partition SMs between the two worker groups) and the pipeline chunk count K (how to split tokens into pipelined microbatches for temporal coordination). The cost model (§3.3) jointly searches over the (c, K) space and, for each candidate pair, derives the number of GEMM tiles n_{steal} that communication workers can steal during their idle periods. The optimal (c^*, K^*) and the corresponding steal count fully determine the execution plan for each layer and GPU.

3.1 Spatial Scheduler

The spatial scheduler determines how the available SMs are divided between communication and computation. As shown in Figure 4(c), the overall latency as a function of the SM split has a clear optimum that depends on the routed workload of the current layer.

As shown in Figure 4(a)(b), both communication bandwidth and GEMM throughput saturate well before all N SMs are assigned; a small number of communication SMs is sufficient to approach peak bandwidth, leaving the majority for computation.

We now formalize the optimization. Let N denote the number of SMs available to the megakernel, and let c denote the number of SMs assigned to communication; the remaining $N - c$ SMs are assigned to computation. Let H denote the hidden dimension, I the per-expert FFN intermediate dimension, and $B = 2H$ bytes the per-token communication size (each token is an H -element BF16 vector, 2 bytes per element). After routing, let X_{local} denote the number of local tokens routed to local experts, X_{in} the number of remote tokens routed to local experts, and $X_{\text{in}}^{\text{uniq}}$ the deduplicated

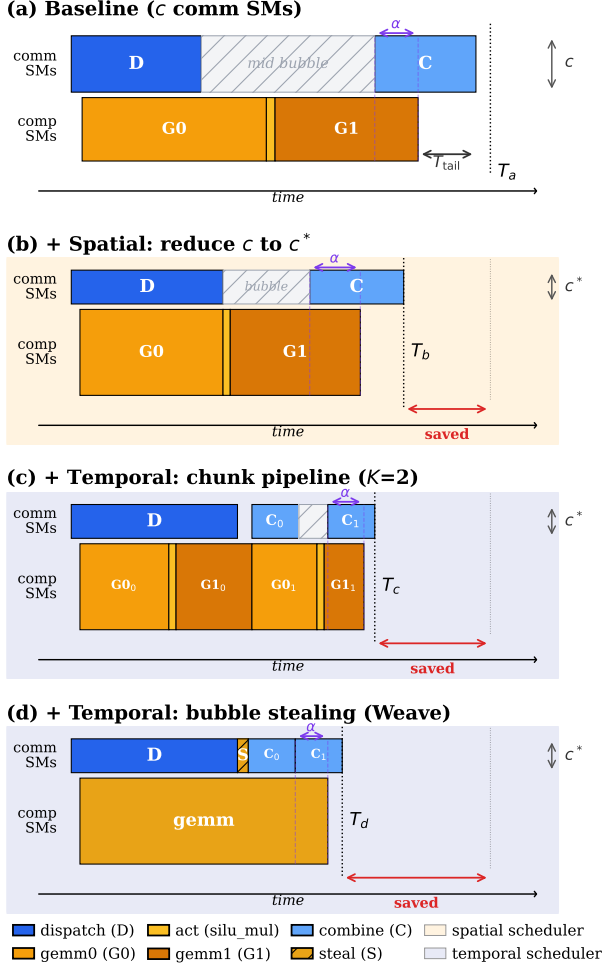


Figure 8. Operational details of the spatial and temporal schedulers. (a) Baseline with communication-computation overlap enabled; a large mid bubble persists. (b) The spatial scheduler applies a near-optimal communication/computation SM ratio, reducing the mid bubble. (c) The temporal scheduler partitions tokens into microbatches, exposing additional overlap opportunities and further reducing the mid bubble. (d) The temporal scheduler fills the residual mid bubble by assigning idle communication SMs to execute GEMM tiles.

count of X_{in} (under top- k routing with $k > 1$, the same remote token may be selected by multiple local experts, so $X_{in}^{unq} \leq X_{in}$; each unique remote token is transferred across the network only once and then reused for all selecting experts [30, 33]). The per-layer workloads are:

$$W_{comp} = (X_{local} + X_{in}) \cdot 6HI \text{ FLOPs}, \quad (1)$$

$$W_{dispatch} = X_{in}^{unq} \cdot B \text{ bytes}, \quad (2)$$

$$W_{combine} = X_{in} \cdot B \text{ bytes}. \quad (3)$$

W_{comp} counts the GEMM work executed jointly on $X_{local} + X_{in}$ token-expert pairs (both local and remote). For communication, the dispatch volume is $X_{in}^{unq} \cdot B$ (each unique remote token is pulled once and reused) and the combine volume is $X_{in} \cdot B$ (one per-expert output returned for each token-expert pair).

We let $\alpha \in [0, 1)$ denote the tile-level overlap ratio between communication and computation, calibrated empirically. The remaining fraction $(1 - \alpha)$ of communication is exposed as a tail outside the overlap window. The three time components are

$$T_{comp} = \frac{W_{comp}}{\text{TFLOPS}(N-c)}, \quad (4)$$

$$T_{comm} = \frac{W_{dispatch} + W_{combine}}{\text{BW}(c)}, \quad (5)$$

$$T_{tail} = (1 - \alpha) \cdot \frac{W_{combine}}{\text{BW}(c)}. \quad (6)$$

The layer is bounded by whichever of the two paths is slower, plus the unhidden tail. The spatial scheduler therefore selects

$$T_{total} = \max(T_{comp} + T_{tail}, T_{comm}), \quad (7)$$

$$c^* = \arg \min T_{total}. \quad (8)$$

If $T_{comp} + T_{tail} > T_{comm}$, the layer time is determined by the GEMM together with the unhidden combine tail; otherwise the GEMM fits entirely within the communication window and the layer time is determined by serialized dispatch and combine.

3.2 Temporal Scheduler

Given the SM partition c from the spatial scheduler, the temporal scheduler addresses how the two groups of SMs coordinate over time. As Figure 8(a) shows, under a fixed SM partition with no temporal scheduling, Dispatch and Combine cluster at the two ends of the timeline, leaving a large stretch of idle comm SMs in the middle of the MoE layer and forming a large *mid bubble*, i.e., an idle interval during the GEMM phase.

To eliminate this bubble, Weave uses two mechanisms: the **chunk pipeline** (§3.2.1) spreads Combine into the middle of the GEMM phase for fine-grained overlap; **bubble stealing** (§3.2.2) lets comm SMs steal GEMM tiles in the residual mid bubble that the chunk pipeline still cannot cover.

3.2.1 Chunk Pipeline.

The mid bubble in Figure 8(a) comes from the DAG dependency: a Combine operation cannot start until the corresponding GEMM1 tile has been produced. To relax this constraint, we split the tokens into smaller chunks; chunks are independent of each other, which opens up more overlap

opportunities. As Figure 8(c) shows, we split GEMM0, SiLU-Mul, GEMM1, and Combine into K chunks, while Dispatch is not chunked, so that communication can overlap with computation as much as possible. Unlike a naive approach where comm SMs switch to comp after dispatch finishes, the chunk pipeline keeps comm SMs continuously doing communication throughout the middle phase. Figure 8(c) illustrates $K=2$: chunk 0’s Combine (C_0) runs during chunk 1’s GEMM, so communication and computation execute concurrently in time.

The choice of K involves a trade-off: a larger K gives finer chunks and a smaller mid bubble, but each chunk holds fewer tokens, degrading per-chunk GEMM throughput. As Figure 9a shows, we benchmark the group GEMM performance under the chunk pipeline on H100 using DSv3 with input seq = 8k. As K increases, the performance degradation accelerates; at $K=8$, throughput drops by $\sim 27\%$ compared to $K=1$. We capture this degradation with a correction factor $\text{eff}_{\text{chunk}}(K) \in (0, 1]$, defined as the ratio of chunked GEMM throughput to unchunked throughput ($K=1$). The computation time under chunking becomes:

$$T_{\text{comp}}(c, K) = \frac{W_{\text{comp}}}{\text{TFLOPS}(N-c) \cdot \text{eff}_{\text{chunk}}(K)}, \quad (9)$$

where $\text{eff}_{\text{chunk}}(K) \in (0, 1]$ is the ratio of chunked GEMM throughput to the unchunked case. Chunking also modifies the tail term: the final chunk’s combine, the only segment not absorbed by a subsequent GEMM, is now of size W_{combine}/K , so the tail is reduced by a factor of K relative to the unchunked case:

$$T_{\text{tail}} = (1 - \alpha) \cdot \frac{W_{\text{combine}}}{\text{BW}(c) \cdot K}. \quad (10)$$

A larger K reduces the unhidden tail but increases the GEMM time through the degradation of $\text{eff}_{\text{chunk}}(K)$; a smaller K does the opposite. The optimum balances these two effects.

3.2.2 Bubble Stealing.

After the joint search of §3.3 chooses (c^*, K^*) , communication SMs may still have idle capacity between the end of dispatch and the start of the chunked combine. We fill this gap by letting them steal GEMM tiles, shown as the S block in Figure 8(d). Once Dispatch finishes, comm SMs join the $N-c$ comp SMs on GEMM work, converting idle capacity into effective GEMM throughput. We place the steal window right after Dispatch so that stolen GEMM tiles execute consecutively and subsequent Combine operations also run without interruption. If stealing were instead scattered into the idle gaps between each chunk’s Combine, both GEMM and Combine would be repeatedly interrupted by role switches, degrading performance. This consolidated placement yields the pattern shown in Figure 8(d).

Below we derive the number of tiles to steal. Given c , communication takes

$$t_{\text{comm}}(c) = \frac{W_{\text{comm}}}{\text{BW}(c)}. \quad (11)$$

During this interval, the $N-c$ comp SMs complete

$$W_{\text{partial}}(c) = t_{\text{comm}}(c) \cdot \text{TFLOPS}(N-c) \quad (12)$$

of GEMM work, leaving

$$W_{\text{steal}}(c) = \max(0, W_{\text{comp}} - W_{\text{partial}}(c)) \quad (13)$$

GEMM FLOPs to be stolen. Once Dispatch finishes, all N SMs are available for GEMM, so $W_{\text{steal}}(c)$ is shared equally across N SMs. Letting W_{tile} denote the FLOPs of a single GEMM tile, each SM steals

$$n_{\text{steal}}(c) = \frac{W_{\text{steal}}(c)}{N \cdot W_{\text{tile}}} \quad (14)$$

GEMM tiles.

3.3 Joint Search for (c^*, K^*)

Substituting the K -aware definitions of T_{comp} and T_{tail} from §3.2 into the total layer time expression of §3.1 yields

$$T_{\text{total}}(c, K) = \max(T_{\text{comp}}(c, K) + T_{\text{tail}}(c, K), T_{\text{comm}}(c)). \quad (15)$$

The scheduler then jointly searches for

$$(c^*, K^*) = \arg \min_{c, K} T_{\text{total}}(c, K). \quad (16)$$

In the comp-dominated regime (if $T_{\text{comp}} + T_{\text{tail}} > T_{\text{comm}}$), increasing K raises T_{comp} through the degradation of $\text{eff}_{\text{chunk}}(K)$ but lowers T_{tail} via the $1/K$ scaling. The two opposing effects produce the real measured (c, K) grid result (Fig. 9b). The number of GEMM tiles to steal is then computed from (c^*, K^*) .

Figure 9b shows the measured latency under different (c, K) configurations. We use DSv2-Lite with input sequence length 8192 for this experiment. The grid exhibits a single clear optimum at $c^*=40, K^*=4$ (1.365 ms), confirming that c and K must be searched jointly.

Once (c^*, K^*) is determined, the scheduler further computes the per-SM steal-tile count $n_{\text{steal}}(c^*)$ via equations (13) and (14) from §3.2.2. At this point the megakernel’s complete execution plan (SM role assignment, chunk count, and the per-SM steal-tile count) is fully determined, and the kernel enters its main execution phase.

4 Implementation

Weave is built on PK [26] and implemented as a single persistent megakernel, launched with a grid size equal to the SM count ($N=132$ on H100). After each layer’s routing completes, the routing result is passed as a kernel parameter; the megakernel then runs the cost model of §3.3 on-GPU to obtain the current layer’s (c^*, K^*) . Both c^* and K^* can therefore switch dynamically per layer.

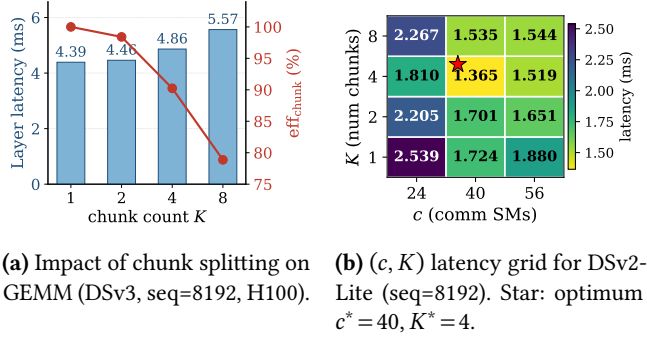


Figure 9. (a) Chunk splitting degrades GEMM throughput; at $K=8$, performance drops by $\sim 27\%$. (b) Measured latency over the (c, K) grid shows a clear unimodal optimum, confirming joint search is necessary.

Each block derives its role from c^* and `blockIdx.x`: blocks with index $< c^*$ are comm SMs (executing dispatch and combine), while the remaining blocks are omp SMs (executing `gemm0`, activation, and `gemm1`). All tile assignments use dynamic scheduling: each SM claims its next tile via `atomicAdd` on a global counter following the chunk pipeline DAG order, naturally tolerating variation in SM execution speeds [27]. After dispatch finishes, comm SMs steal n_{steal} GEMM tiles before transitioning to combine. For the final combine, all N SMs participate regardless of role, eliminating tail idleness.

Weave also fuses dispatch-side token deduplication [30] into the megakernel. DeepEP [30] and TD [32] perform deduplication as a separate operation before GEMM0, whereas Weave overlaps dispatch with GEMM0 and achieves deduplication naturally during dispatch. After routing completes, duplicate tokens are identified and each comm tile knows in advance whether its token is a duplicate. If so, the tile reads directly from a local HBM buffer populated by the first transfer, avoiding both redundant cross-GPU transfers and HBM polling.

5 Evaluation

5.1 Evaluation Setup

Hardware. All experiments are conducted on a node with $4 \times$ NVIDIA H100 80 GB SXM5 GPUs. Each GPU has 132 SMs, 80 GB HBM3, and a peak BF16 throughput of 989 TFLOPS. The GPUs are fully connected through NVLink and NVSwitch, with 450 GB/s unidirectional bandwidth per GPU. The host has two Intel Xeon Platinum 8468 CPUs, each with 48 physical cores and 96 hardware threads, for a total of 96 cores and 192 hardware threads across 2 NUMA nodes. The system has 2.0 TiB of CPU memory.

Software. We use CUDA Toolkit 12.9.86, NVIDIA driver 575.57.08, NCCL 2.21.5, NVSHMEM 3.6.5, Python 3.10.12,

Table 1. MoE model configurations used in evaluation.

Model	E	$\text{top-}k$	H	I
DSv3	256	8	7168	2048
Phi-3.5-MoE	16	2	4096	6400
Qwen3-30B	128	8	2048	768
Qwen3.5-35B	256	8	2048	512
DSv2-Lite	64	6	2048	1408
DSv2	160	6	5120	1536

and PyTorch 2.6.0+cu124. The evaluated baseline implementations are SGLang v0.5.9, Triton-Distributed(TD) v3.4.0, DeepEP v1.2.1, Comet with Flux v1.1.2, and ParallelKittens at commit a8f63a9.

Models. We evaluate six mainstream BF16 MoE architectures under expert parallelism with $\text{EP}=4$, as summarized in Table 1. Here, E denotes the total number of routed experts, $\text{top-}k$ denotes the number of experts selected per token, H denotes the hidden dimension, and I denotes the per-expert intermediate dimension.

Workloads. We sample routing inputs from the ShareGPT dataset and control the input sequence length in $\{2048, 4096, 8192\}$ with batch size 1.

Baselines. We compare Weave with five representative MoE serving and communication-computation overlap systems. SGLang [31] is a production-grade serving system that uses NCCL All-to-All and cuBLAS grouped GEMM. TD [32] fuses dispatch with GEMM0 and GEMM1 with combine, and uses global task preemption within fused operators to reduce inter-operator bubbles. DeepEP [30] is an expert-parallel communication library that chunks tokens into pipeline stages to overlap dispatch and combine with expert GEMM; we use DeepGEMM [29] as its GEMM backend and keep its default communication partition of 20 SMs. Comet [28] fuses dispatch with GEMM0 and GEMM1 with combine, enabling tile-level communication-computation overlap. ParallelKittens (PK) [26] fuses dispatch with expert up-projection under a compile-time fixed SM partition.

5.2 Per-Layer MoE Latency

We compare Weave against five baselines (SGLang [31], PK [26], TD [32], DeepEP [30], Comet [28]) across 6 MoE architectures and 3 sequence lengths (2k, 4k, 8k tokens). As shown in Figure 10 (Row 1), Weave achieves the lowest per-layer latency in all 18 configurations.

Weave achieves $1.95\times$ – $4.76\times$ geometric mean speedup over all baselines: vs. DeepEP $1.95\times$, vs. Comet $2.01\times$, vs. TD $2.95\times$, vs. SGLang $3.63\times$, vs. PK $4.76\times$. DeepEP uses DeepGEMM [29] as its GEMM backend and chunks tokens into pipeline stages to overlap dispatch/combine with expert GEMM. Comet performs tile-level overlap within two fused kernels (dispatch+gemm0 and gemm1+combine) and can select the communication SM count per iteration

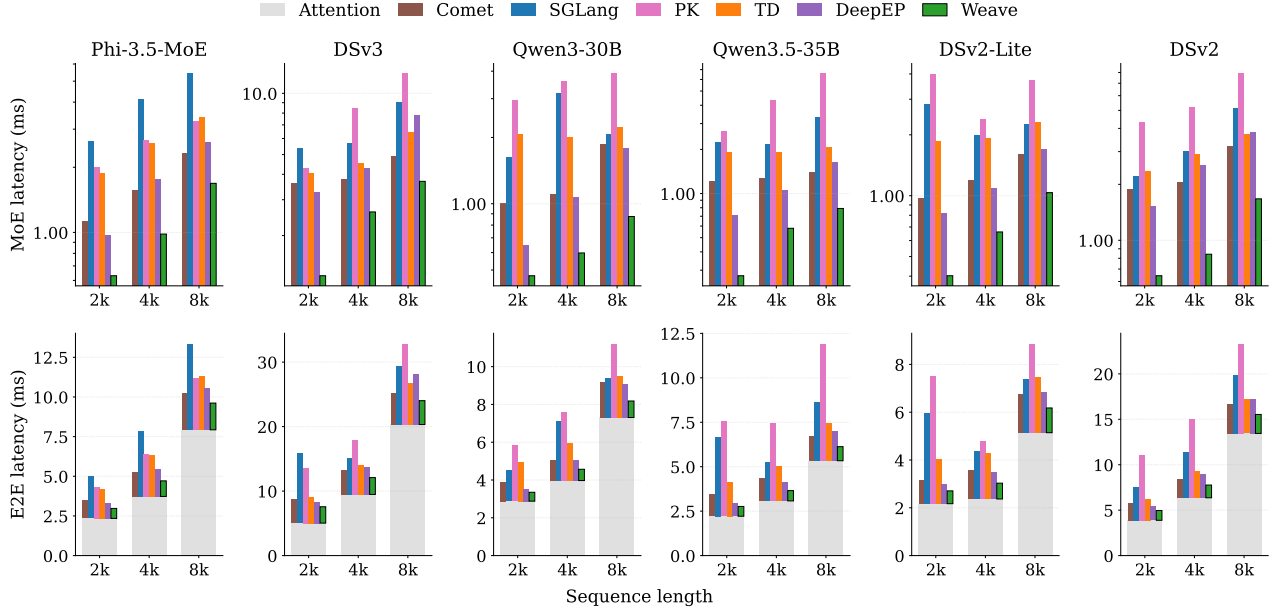


Figure 10. Per-layer MoE latency and end-to-end latency on 4xH100 SXM, EP=4, BF16. **Row 1:** per-layer MoE latency (log-scale y-axis) across six models and three sequence lengths (2k/4k/8k). **Row 2:** end-to-end latency breakdown for the same six models at the same three sequence lengths; the gray band is attention layer time and the colored bars on top are MoE layer time for each system.

based on sequence length. TD fuses the same two operator pairs into two kernels, but within each fused kernel communication and computation run serially across all SMs, with only marginal overlap at operator boundaries via global preemption. PK fuses dispatch with the expert up-projection under a compile-time fixed SM partition. SGLang issues separate NCCL and cuBLAS calls without kernel-level overlap. None of these baselines adapts the SM partition to per-layer routing results.

5.3 End-to-End Performance

We evaluate end-to-end (E2E) latency by combining the pre-MoE/attention block time measured via SGLang with the MoE kernel time from Weave and each baseline. We test the same 6 MoE architectures at sequence lengths 2048, 4096, and 8192 (batch size 1, 4xH100 EP=4), yielding 18 configurations per baseline. Figure 10 (Row 2) shows the E2E latency breakdown for all 18 configurations. The gray band represents the attention time and the colored bars show each system’s MoE time.

Weave achieves the lowest E2E latency in all 18 configurations, with $1.12\times$ – $1.70\times$ geometric mean E2E speedup across baselines: vs. DeepEP $1.12\times$, vs. Comet $1.13\times$, vs. TD $1.28\times$, vs. SGLang $1.50\times$, vs. PK $1.70\times$. The E2E advantage is more pronounced at shorter sequence lengths: as sequence length grows, attention time scales quadratically while MoE time scales linearly, so the attention portion increasingly dominates E2E latency and dilutes Weave’s MoE-layer speedup.

5.4 Hardware Utilization Profiling

We profile Weave and other baselines (TD, Comet, DeepEP, PK) with Nsight Systems on DSv2-Lite (seq = 2048, 4xH100 SXM, EP=4). The profiling window covers the complete MoE forward pass from the end of routing to the end of combine. We report three GPU metrics (SM Active, NVLink Utilization, Tensor Core HMMA) and a derived overlap ratio, defined as the fraction of the profiling window during which both NVLink Utilization and Tensor Core HMMA are simultaneously greater than zero.

Figure 11 summarizes the mean (bar) and peak (hollow circle atop dashed line) of the three metrics over each system’s MoE forward window, together with the overlap ratio. Weave achieves the highest overlap ratio (47.1%). The remaining baselines fall well below: Comet (14.3%), DeepEP (9.2%), PK (5.8%), and TD (5.1%). Comet has the second-highest SM Active mean (47.4%) because its two-segment fusion keeps SMs busy, but its overlap is moderate. DeepEP’s low overlap stems from a mismatched communication SM count: NVLink activity appears as brief bursts that quickly saturate and stop, leaving the vast majority of the timeline with only Tensor Core active.

Weave achieves leading performance across all four metrics. *SM Active*: fusing all five MoE operations into a single persistent megakernel eliminates inter-kernel launch gaps, and bubble stealing ensures communication SMs have GEMM tiles to execute after their primary work finishes,

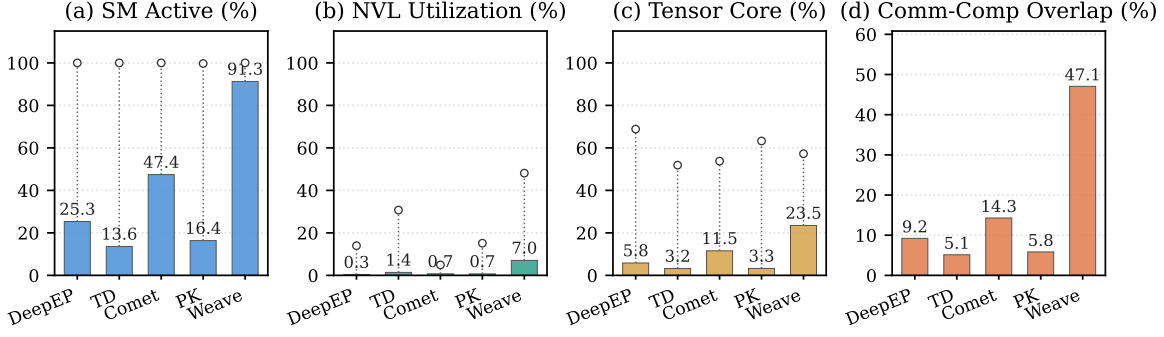


Figure 11. Aggregate GPU metrics over the MoE forward window (DSv2-Lite, seq = 2048, 4×H100, EP=4). Bars: mean; hollow circles atop dashed lines: peak. (a) SM Active (%). (b) NVLink Utilization (%). (c) Tensor Core HMMA (%). (d) Comm-comp overlap ratio (%).

keeping all SMs occupied throughout the layer. *NVLink Utilization* and *Tensor Core HMMA*: the spatial scheduler maintains a well-matched communication/computation SM ratio throughout the kernel’s execution, so neither NVLink bandwidth nor compute throughput starves while the other is saturated. *Overlap ratio*: the chunk pipeline extends the communication window from the head and tail of the layer to cover the entire computation phase by interleaving combine with gemm over time, maximizing the duration where NVLink and Tensor Core are simultaneously active.

5.5 Cost Model Accuracy

We evaluate the cost model along two dimensions: selection accuracy and online cost. As shown in Figure 12, we sweep 4 models × 4 sequence lengths on 4×H100 SXM (EP=4), each with 54 (c, K) configurations. **(a) Selection accuracy.** For each (model, seq_len) group, we compare the measured latency at the cost-model-selected (c, K) against the measured latency at the exhaustive-search optimum. The average accuracy gap is 8.2%. **(b) Online cost.** Because the cost model is fused inside the megakernel, each block computes independently and only reads a small amount of routing results from global memory. On DSv3, the cost model overhead is a nearly constant $0.54 \mu s$ regardless of sequence length, less than 0.021% of the MoE layer time, indicating that the run-time search adds negligible overhead.

5.6 Design Ablation

We isolate the contributions of the spatial and the temporal scheduler by comparing three configurations on Qwen3-30B and Qwen3.5-MoE at sequence lengths 2k, 4k, 8k, and 16k (4×H100, EP=4).

- Weave w/o S+T: both schedulers disabled; dispatch / gemm / combine run sequentially on all SMs (no compute-communication overlap).

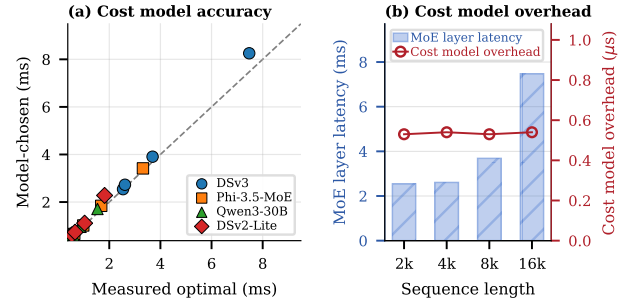


Figure 12. Cost model evaluation. (a) Configuration-selection quality: measured-optimal latency vs. measured latency at the cost-model-chosen (c, K); dashed line is $y=x$. (b) Online overhead on DSv3: MoE layer latency (left blue axis, bars) vs. cost model overhead (right red axis, hollow markers).

- Weave w/o T: spatial scheduler (S) enabled, temporal scheduler (T) disabled (no chunk pipeline or bubble stealing). Isolates the spatial scheduler’s contribution.
- Weave: full design with both spatial and temporal schedulers.

As shown in Figure 13, going from Weave w/o S+T to Weave w/o T quantifies the gain of explicit SM partitioning; going from Weave w/o T to Weave further quantifies the additional gain of the temporal scheduler. Both design components yield significant positive gains at every (model, seq_len) point. On Qwen3-30B at seq=4096, enabling S reduces latency by 14.4%, and further enabling T reduces it by 15.7% on top of that; the two stages compose into a 27.8% latency reduction over the no-scheduler configuration. Averaged over the eight (model, seq_len) points, S contributes a 10.1% reduction and T an additional 14.1%, for a total 22.8%

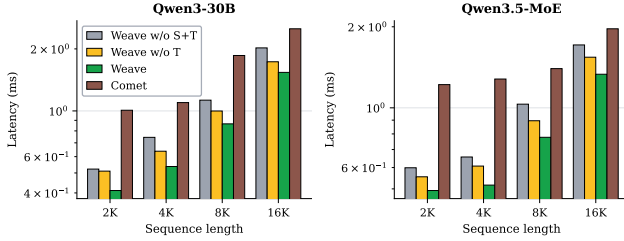


Figure 13. Design ablation on Qwen3-30B and Qwen3.5-MoE (4×H100, EP=4). S = spatial scheduler, T = temporal scheduler. Three configurations: Weave w/o S+T (no overlap), Weave w/o T (spatial only), Weave (full).

reduction. This confirms that both schedulers are effective, and their gains stack.

6 Related Work

Compute-communication overlap. Existing MoE inference systems have extensively studied how to overlap inter-GPU communication with expert computation [4, 7, 9, 13, 16, 23, 24, 34]. Existing systems improve overlap at progressively finer granularity. Host- and stream-level approaches pipeline all-to-all communication with expert computation through operator scheduling and pipeline partitioning [9, 11, 23, 24], but the overlap is still bounded by kernel-level stages. Finer-grained approaches such as FlashOverlap [10] and TokenWeave [8] exploit tile- or token-level opportunities, yet expert computation often remains encapsulated in opaque library kernels, limiting how communication can be interleaved with computation. Kernel fusion addresses this limitation by moving communication into expert GEMM kernels, allowing dispatch or combine to overlap with GEMM at a finer in-kernel granularity rather than only across separate kernel launches. ParallelKittens [26] fuses dispatch with the expert up-projection under a compile-time fixed SM partition, while Comet [28] and TD [32] fuse dispatch/up-projection and down-projection/combine as separate stages. These fused pipelines enable finer-grained overlap by moving communication closer to GEMM execution, but remain limited by fixed SM partitioning, partial fusion, or residual launch boundaries between fused stages.

Persistent megakernel. Fusing all operators into a single persistent megakernel has gained increasing attention [1, 2, 5, 14, 17, 19, 25, 33, 35]. MoE kernel fusion reduces launch overhead and exposes opportunities for cross-operator scheduling. Partial-fusion approaches still retain inter-operator boundaries, while whole-layer megakernels such as FlashDMoE [1] integrate dispatch, expert computation, and combine into a single kernel. Whole-layer fusion removes launch boundaries and creates a larger scheduling domain. For highly dynamic workloads

such as MoE, effectively using this domain requires runtime control over task ordering and resource allocation. MPK [2] introduces scheduler SMs for device-side task scheduling and also analyzes MoE workloads, but its MoE support is limited to single-GPU execution without cross-GPU dispatch and combine communication. When MPK does consider multi-GPU communication, it targets only static tensor parallelism (TP), thereby missing the compute-communication overlap opportunities unique to expert parallelism where routing-dependent workloads vary per layer. TD’s piecewise megakernel enables limited scheduling within fused regions, but still only partially fuses MoE operators and does not jointly adapt SM partitioning and intra-layer execution order after routing. Weave addresses this gap by using a persistent megakernel as a routing-aware scheduling domain that adapts both decisions at runtime.

7 Conclusion

We present Weave, to our knowledge the first MoE overlap system whose SM split is decided per layer and per GPU by routing results at runtime. Existing systems either fix the SM partition at compile time, adjust it only at coarse granularity, or avoid explicit SM partitioning and run communication and computation serially with minimal overlap; Weave instead exploits the fact that each layer’s communication and computation volumes become known after routing, running a lightweight cost model inside the persistent megakernel to jointly select the communication SM count and chunk pipeline configuration online. On 4×H100 SXM GPUs across six mainstream MoE models, Weave achieves a 2.89× geometric-mean MoE-layer speedup and a 1.33× geometric-mean end-to-end speedup over five state-of-the-art baselines.

Limitations and future work. Due to hardware constraints, Weave is currently validated on a single 4×H100 NVLink node with EP=4. Extending to larger GPU counts and multi-node configurations is left for future work.

References

- [1] Osayamen Jonathan Aimuyo, Byungsoo Oh, and Rachee Singh. 2025. FlashMoE: Fast Distributed MoE in a Single Kernel. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2506.04667.
- [2] Xinhao Cheng, Zhihao Zhang, Yu Zhou, Jianan Ji, Jincheng Jiang, Zepeng Zhao, Ziruo Xiao, Zihao Ye, Yingyi Huang, Ruihang Lai, Hongyi Jin, Bohan Hou, Mengdi Wu, Yixin Dong, Anthony Yip, Songting Wang, Wenqin Yang, Xupeng Miao, Tianqi Chen, and Zhihao Jia. 2025. Mirage Persistent Kernel: A Compiler and Runtime for Megakernelizing Tensor Programs. *arXiv preprint arXiv:2512.22219* (2025).
- [3] Damai Dai, Chengqi Deng, Chenggang Zhao, R.X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y.K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

- [4] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).
- [5] Greg Diamos, Shubho Sengupta, Bryan Catanzaro, Mike Chrzanowski, Adam Coates, Erich Elsen, Jesse Engel, Awni Hannun, and Sanjeev Satheesh. 2016. Persistent RNNs: Stashing Recurrent Weights On-Chip. *ICML* (2016).
- [6] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research (JMLR)* 23, 120 (2022), 1–39.
- [7] Yunqi Gao, Bing Hu, Mahdi Boloursaz Mashhadi, A-Long Jin, Yanfeng Zhang, Pei Xiao, Rahim Tafazolli, and M  rouane Debbah. 2025. FlowMoE: A Scalable Pipeline Scheduling Framework for Distributed Mixture-of-Experts Training. In *Advances in Neural Information Processing Systems (NeurIPS)*. *arXiv:2510.00207*.
- [8] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. 2025. TokenWeave: Efficient Compute-Communication Overlap for Distributed LLM Inference. *arXiv preprint arXiv:2505.11329* (2025).
- [9] Jiaao He, Jidong Zhai, Tiago Antunes, Haojun Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. FasterMoE: Modeling and Optimizing Training of Large-Scale Dynamic Pre-Trained Models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*.
- [10] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2025. FlashOverlap: A Lightweight Design for Efficiently Overlapping Communication and Computation. *arXiv preprint arXiv:2504.19519* (2025).
- [11] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, HoYuen Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. In *Proceedings of Machine Learning and Systems (MLSys)*. *arXiv:2206.03382*.
- [12] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024).
- [13] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. 2024. Lancet: Accelerating Mixture-of-Experts Training via Whole Graph Computation-Communication Overlapping. In *Proceedings of Machine Learning and Systems (MLSys)*. *arXiv:2404.19429*.
- [14] Hongyi Jin, Bohan Hou, Guanjie Wang, Ruihang Lai, Jinqi Chen, Zihao Ye, Yaxing Cai, Yixin Dong, Xinhao Cheng, Zhihao Zhang, Yilong Zhao, Yingyi Huang, Lijie Yang, Jincheng Jiang, Gabriele Oliaro, Jianan Ji, Xupeng Miao, Vinod Grover, Todd C Mowry, Zhihao Jia, and Tianqi Chen. 2025. EventTensor: A Unified Abstraction for Compiling Dynamic Megakernel. *arXiv preprint arXiv:2604.13327* (2025).
- [15] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations (ICLR)*.
- [16] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating Distributed MoE Training and Inference with Lina. In *USENIX Annual Technical Conference (ATC)*.
- [17] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *OSDI*.
- [18] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. 2025. OLMoE: Open Mixture-of-Experts Language Models. In *International Conference on Learning Representations (ICLR)*.
- [19] Aniruddha Nrusimha, William Brandon, Mayank Mishra, Yikang Shen, Rameswar Panda, Jonathan Ragan-Kelley, and Yoon Kim. 2025. FlashFormer: Whole-Model Kernels for Efficient Low-Batch Inference. *arXiv preprint arXiv:2505.22758* (2025).
- [20] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
- [21] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>. Flagship MoE: Qwen3.5-397B-A17B (397B total / 17B activated, 512 routed + 1 shared expert, top-10).
- [22] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*.
- [23] Shaohuai Shi, Xinglin Pan, Xiaowen Chu, and Bo Li. 2023. PipeMoE: Accelerating Mixture-of-Experts through Adaptive Pipelining. In *IEEE Conference on Computer Communications (INFOCOM)*.
- [24] Shaohuai Shi, Xinglin Pan, Qiang Wang, Chengjian Liu, Xiaozhe Ren, Zhongzhe Hu, Yu Yang, Bo Li, and Xiaowen Chu. 2024. ScheMoE: An Extensible Mixture-of-Experts Distributed Training System with Tasks Scheduling. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys)*. 236–249. doi:10.1145/3627703.3650083
- [25] Benjamin Spector, Jordan Juravsky, Stuart Sul, Owen Dugan, Dylan Lim, Dan Fu, Simran Arora, and Christopher R  . 2025. Look Ma, No Bubbles! Designing a Low-Latency Megakernel for Llama-1B. <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>. Hazy Research blog post.
- [26] Stuart H. Sul, Simran Arora, Benjamin F. Spector, and Christopher R  . 2025. ParallelKittens: Systematic and Practical Simplification of Multi-GPU AI Kernels. *arXiv preprint arXiv:2511.13940* (2025).
- [27] Renji Thomas, Kristin Barber, Naser Sedaghati, Li Zhou, and Radu Teodorescu. 2016. Core Tunneling: Variation-Aware Voltage Noise Mitigation in GPUs. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 151–162. doi:10.1109/HPCA.2016.7446061
- [28] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. In *Proceedings of Machine Learning and Systems (MLSys)*. *arXiv:2502.19811*.
- [29] Chenggang Zhao, Zhean Xu, Liang Zhao, Jiaoshi Li, Chenhao Xu, Anyi Xu, Shengyu Liu, Kexing Zhou, and Kuai Yu. 2025. DeepGEMM: clean and efficient BLAS kernel library on GPU. <https://github.com/deepseek-ai/DeepGEMM>.
- [30] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiaoshi Li, and Liang Zhao. 2025. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>.
- [31] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. *arXiv:2312.07104*.
- [32] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. *arXiv:2504.19442 [cs.DC]* <https://arxiv.org/abs/2504.19442>
- [33] Size Zheng, Xuegui Zheng, Li-Wen Chang, and Jidong Zhai. 2025. UniEP: Unified Expert-Parallel MoE MegaKernel for LLM Training. *arXiv preprint arXiv:2604.19241* (2025).

- [34] Guichao Zhu, Lintian Lei, Yuhao Qing, Yichao Fu, Fanxin Li, Dong Huang, Zekai Sun, and Heming Cui. 2025. FoldMoE: Efficient Long Sequence MoE Training via Attention-MoE Pipelining. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 3705–3717. doi:10.18653/v1/2025.acl-long.186
- [35] Donglin Zhuang, Zhen Zheng, Haojun Xia, Xiafei Qiu, Junjie Bai, Wei Lin, and Shuaiwen Leon Song. 2024. MonoNN: Enabling a New Monolithic Optimization Space for Neural Network Inference Tasks on Modern GPU-Centric Architectures. In *OSDI*.